Solving Wave Equation using Finite Element Method

[1]Aliyu Bhar Kisabo and [2]James Musa Ibrahim
Center for Space Transport & Propulsion Epe, Lagos-State
[1]aliyukisabo@gmail.com, [2]jamescap02@yahoo.com

Introduction

In mathematics, a **hyperbolic partial differential equation** of order n is a partial differential equation (PDE) that, roughly speaking, has a well-posed initial value problem for the first $n-1$ derivatives. More precisely, the Cauchy problem can be locally solved for arbitrary initial data along any non-characteristic hypersurface. Many of the equations of mechanics are hyperbolic, and so the study of hyperbolic equations is of substantial contemporary interest [1]. The model hyperbolic equation is the wave equation and the most general form of the one-dimensional wave equation [2] is given in (1)

$$\frac{\partial^2 u}{\partial t^2} + \xi \frac{\partial u}{\partial t} + ku = c^2 \frac{\partial^2 u}{\partial x^2} + F(x,t) \quad (1)$$

Where c is the wave speed and ξ is the damping coefficient. k is the external restoration factor. $F(x, t)$ is the arbitrary external forcing function.

Damped Wave Equation

Consider a piece of thin flexible string of length L of negligible weight. Suppose the two ends of the string are firmly secured (clamped) at some supports so they will not move. Assume the set-up has damping. Then, the vertical displacement of the string, $0 < x < L$ and at any time $t > 0$, is given by the displacement function $u(x,t)$. It satisfies the one-dimensional damped wave equation [3,4] given in (2)

$$\frac{\partial^2 u}{\partial t^2} + \xi \frac{\partial u}{\partial t} = c^2 \frac{\partial^2 u}{\partial x^2} \quad (2)$$

Here, $\xi \neq 0$ and friction is due to the viscosity of medium and density of string is being considered in (2). Also, (2) is homogeneous ($F = 0$)

With boundary conditions: $u(0,t) = 0, \quad u(L,t) = 0, \quad t > 0$

Initial conditions: $u(x,0) = f(x), \quad \frac{\partial u}{\partial t}(x,0) = g(x), \ 0 < x < L$

The Finite Element Method (FEM)

The *finite element method* is a numerical method generally used to solve differential equations with boundary conditions. The concept of the finite element method is to divide the domain into finitely many smaller areas called elements and approximate the solution of the differential equation on these elements using a suitable set of basis functions.

The FEM has a few advantages over other numerical methods for solving differential equations (like the finite difference method). Firstly, the finite element method approximates the solution for almost the whole domain, while most other methods only approximate the solution at a discrete set of points. Secondly, with the finite element method, we have more freedom over the discretization of the domain. We have the choice to increase precision at parts of the domain by using smaller elements and to decrease the precision at parts by using larger elements. Thirdly we can choose different basis functions on the elements to increase or decrease

the precision at different parts of the domain or choose basis functions which better approximate the actual solution, for example by using trigonometric functions instead of polynomials.

Thus, the finite element method as a general tool for the numerical solution of two-point boundary value problems. In doing so, the basic idea is to first rewrite the boundary value problem as a variational equation, and then seek a solution approximation from the space of continuous piecewise linear. This discretization procedure results in a linear system that can be solved on a computer [5].

Solution

We can simply re-write (2) as given in (3)

$$\ddot{u}_{tt} + \xi\dot{u} = c^2 u_{xx} \qquad (3)$$

With initial boundary conditions as

$$u(0,t) = u(1,t) = 0 \quad \text{for all } t \in \mathbb{R}$$

$$u(x,0) = u_0(x) \qquad \text{for all } x \in [0,1] \text{ and} \qquad (3a)$$

$$\frac{\partial u}{\partial t}(x,0) = u_1(x) \qquad \text{for all } x \in [0,1]$$

The friction term ξ opposes the motion of string and means that eventually vibrations decay with time

To solve (3) using the FEM, the following will be done:

follows:

- Derive a "weaker" formulation of the differential equation,
- Define the spatial discretization of the domain into elements and nodes,
- Define the basis functions for the approximation,
- Derive a system of equations,
- Solve the system of equations.

Weak formulation

Let $v(x)$ be a differentiable function such that $v(0) = v(1)=0$ and therefore zero on the boundaries of the domain. By multiplying the wave equation with this function and integrating it over the unit interval we get following expression in (4)

$$0 = \int_0^1 \left(\frac{\partial^2 u}{\partial t^2}(x,t) + \xi \frac{\partial u}{\partial t}(x,t) - c^2 \frac{\partial^2 u}{\partial x^2}(x,t) \right) v(x)\, dx. \quad (4)$$

Integrating (4) by part will give us (5)

$$with\ a(u(x,t), v(x)) = \int_0^1 \frac{\partial^2 u}{\partial t^2}(x,t) v(x)\, dx + \xi \int_0^1 \frac{\partial u}{\partial t}(x,t) v(x)\, dx + c^2 \int_0^1 \frac{\partial u}{\partial x}(x,t) \frac{\partial v}{\partial x}(x,t)\, dx. \quad (5)$$

For the integrals above to exist we require certain properties of u, v and their derivatives. For this we will use the L_2-norm on a domain D. For a function f this is defined by (6)

$$\|f\|_2 = \int_D |f(x)|^2 dx. \quad (6)$$

Using this *norm* a function f is called square-integrable if the L_2-norm of f is finite or more specific as given in (7)

$$\|f\|_2 < \infty \quad (7)$$

The set of all square-integrable functions is denoted by L_2. An important property of square-integrable functions is that the integral of the product of two square-integrable functions over the domain D is also finite. This property is important for the existence of the integrals in (5)

We want the functions u, v and their first derivatives concerning x to be square-integrable. The set of functions which are square-integrable and whose first derivative is square-integrable is called a *first-order Sobelov space* and can be denoted by H^1. The set of functions which are also zero at the boundary of the domain is denoted by H_0^1.We now want to find a function u which is in H_0^1 for each moment in time t such that for every function v in H_0^1 the equations of (5) hold.
This gives rise to the weak formulation of our one-dimensional wave equation, which can now be stated as Follows:

Find $u(x,t)$ such that for all $t \in \mathbb{R}$ it holds that $u(\cdot,t) \in H_0^1$ and for $v \in H_0^1$

$$a\left(u(x,t), v(x)\right) = 0 \qquad (8)$$

And the initial conations in (3a) holds

This is called the weak formulation as the function u only has to hold in (8) concerning certain functions v called *test functions*.

Nodes and Elements

Dividing the interval [0,1] into N elements of equal size separated by nodes. The size of each element is given by $h = \frac{1}{N}$, therefore the nodes can be defined as

$$x_i = (i-1)h \quad \text{for} \quad i = 1,\ldots,N+1, \qquad (9)$$

and the *elements* are defined as the intervals (bounded by two nodes)

$$E_i = [x_i, x_{i+1}] \quad \text{for} \quad i = 1,\ldots,N. \qquad (10)$$

The nodes and elements are illustrated in Fig. 1

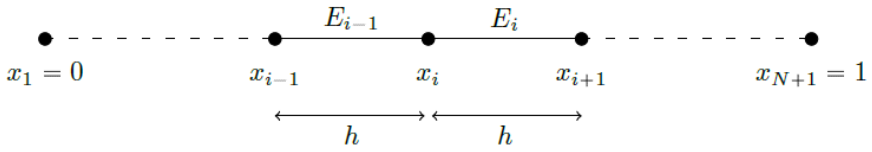


Fig 1: A regular partitioning of the unit interval [0,1] in elements and nodes.

Basis functions

Instead of searching for a function u in H_0^1 such that for all test functions v in H_0^1 the equation of the weak formulation hold, we are going to search in a finite-dimensional subspace of H^1. This space will consist of those functions in H^1 which are piecewise polynomial on each element. We will denote this finite-dimensional set of functions as S^1.

The set S^1 will be defined as the functions v that can be written as a linear combination of finitely many basis functions and that is zero at the boundaries. We will take $N + 1$ basis functions, one for each node, and call these $\phi_i(x)$ for $i = 1,\ldots,N + 1$. Then the functions v in S^1 can be written as given in (11)

$$v(x) = \sum_{i=1}^{N+1} v_i \phi_i(x), \quad \text{with } v_i \in \mathbb{R} \text{ and}$$
$$v(0) = v(1) = 0. \qquad (11)$$

To get piecewise linear functions on the elements the basis functions are defined as follows: For $j = 2,\ldots,N$ we will define a basis function which is non-zero and linear on the elements E_{j-1} and E_j and zero everywhere else. These basis functions can be written as given in (12)

$$\phi_j(x)=\begin{cases}\dfrac{x-x_j-1}{h} & x_{j-1}\le x\le x_j\\ \dfrac{x_{j+1}-x}{h} & x_j\le x\le x_{j+1},\ \text{for } j=2,\ldots,N\\ 0 & \text{otherwise}\end{cases} \qquad (12)$$

For j = 1, N+1 we define basis functions that lie on the boundary of the interval, such that they are non-zero on the elements E_1 and E_N respectively and zero everywhere else. These functions can be written as given in (13)

$$\phi_1(x)=\begin{cases}\dfrac{x_2-x}{h} & x_1\le x\le x_2\\ 0 & otherwise\end{cases}$$
$$\phi_{N+1}(x)=\begin{cases}\dfrac{x-x_N}{h} & x_N\le x\le x_{N+1}\\ 0 & otherwise\end{cases} \qquad (13)$$

These basis functions are illustrated in Fig. 2. An important property of these basis functions is that $\phi_i(x)$ is precisely one at node x_i and zero on every other node, therefore (14) is given.

$$\phi_1(x)=\begin{cases}1 & when\quad i=j\\ 0 & when\quad i\neq j\end{cases} \qquad (14)$$

This results in the property $v(x_i)=v_i$ for $i=1,\ldots,N+1$

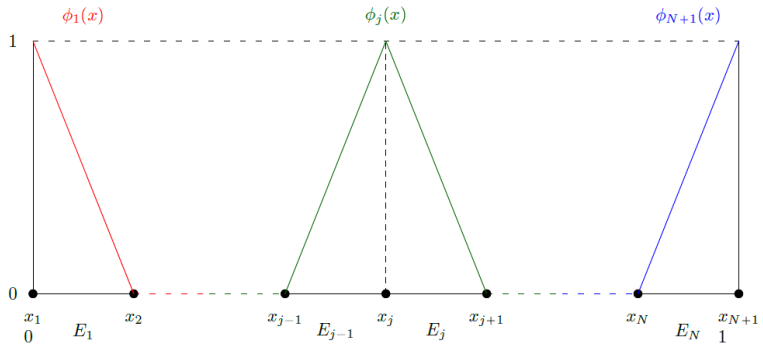


Fig. 2: The piecewise linear basis functions ϕ_1, ϕ_j and ϕ_{N+1}

Finite element solution

Using the just defined basis functions we can define a *finite element solution* and *finite element test functions* as given in (15) and (16) respectively

$$U(x,t)=\sum_{i=1}^{N+1}U_i(t)\phi_i(x) \qquad (15)$$

$$V(x)=\sum_{i=1}^{N+1}V_i\phi_i(x) \qquad (16)$$

Instead of searching for a solution in H^1 we will search in S^1. A finite element formulation can then be formulated as follows

Find $u(x,t)$ such that for all $t\in\mathbb{R}$ it holds that $u(\cdot,t)\in S^1$ and for $V\in S^1$

$$a\left(U\left(x,t\right),\mathrm{V}\left(x\right)\right)=0 \qquad (17)$$

and the following initial boundary conditions hold

$$U\left(0,t\right)=U\left(1,t\right)=0 \quad \text{for all } t\in\mathbb{R}$$
$$U\left(x,0\right)=U_{0,0}\left(x\right) \qquad \text{for all } x\in\left[0,1\right] \text{ and } \quad (17a)$$
$$\frac{\partial U}{\partial t}\left(x,0\right)=U_{0,1}\left(x\right) \qquad \text{for all } x\in\left[0,1\right]$$

Note that because of the form U takes, it does not have to hold that $U_0(x)$ equals $u_0(x)$ and initially we will choose $U_0(x)$ and $U_1(x)$ such that for each node x_i it holds that $U_i(x_i) = U_{0,0}(x_i)$ and $\frac{\partial U_i}{\partial t}\left(0\right)=U_{0,1}\left(x_i\right)=u_1\left(x_i\right)$.

For the finite element solution, it is enough to test it only against the basis functions. This follows from the linearity of $a(u,v)$ in the second variable and therefore it holds that

$$\begin{aligned}0&=a\left(U\left(x,t\right),V\left(x\right)\right)=a\left(U\left(x,t\right)\sum_{i=1}^{N+1}V_i\phi_i\left(x\right)\right)\\&=\sum_{i=1}^{N+1}V_i a\left(U\left(x,t\right),\phi_i\left(x\right)\right).\end{aligned} \qquad (18)$$

Thus, if (18) holds for all of the basis functions ϕ_i, it also holds for all functions $V(x)$ in S^1. For each basis function $\phi_i\left(x\right)$ that is not on the edge with I = 2,...,N we can write (19)

$$\begin{aligned}0&=a(U(x,t),\phi_j(x))\\&=\int_0^1\left(\sum_{i=1}^{N+1}\frac{\partial^2}{\partial t^2}U_i(t)\phi_i(x)\phi_j(x)\right)dx+c^2\int_0^1\left(\sum_{i=1}^{N+1}U_i(t)\frac{\partial}{\partial x}\phi_i(x)\frac{\partial}{\partial x}\phi_j(x)\right)dx\\&=\sum_{i=1}^{N+1}\frac{\partial^2}{\partial t^2}U_i(t)\int_0^1\phi_i(x)\phi_j(x)dx+c^2\sum_{i=1}^{N+1}U_i(t)\int_0^1\frac{\partial}{\partial x}\phi_i(x)\frac{\partial}{\partial x}\phi_j(x)dx\\&=\sum_{i=1}^{N+1}\ddot{U}_i(t)T_{i,j}+c^2\sum_{i=1}^{N+1}U_i(t)S_{i,j},\\&\text{with}\quad T_{i,j}:=\int_0^1\phi_i(x)\phi_j(x)dx\quad\text{and}\quad S_{i,j}:=\int_0^1\frac{\partial}{\partial x}\phi_i(x)\frac{\partial}{\partial x}\phi_j(x)dx.\end{aligned} \qquad (19)$$

This results in N - 1 linear ordinary differential equations. For the two basis functions left out, we can use the following additional equations

$$\ddot{U}_1\left(t\right)=0 \quad \text{and } \ddot{U}_{N+1}\left(t\right)=0 \qquad (20)$$

to make sure the boundary conditions hold.

By defining the vector $U\left(t\right)=\left(U_1\left(t\right),\cdots,U_{N+1}\left(t\right)\right)$ we can use these N +1 linear differential equations to write (21)

$$T\ddot{U}\left(t\right)+\xi\dot{U}\left(t\right)+c^2SU\left(t\right)=0 \qquad (21)$$

with the coefficients of T and S being those derived in (19) except for the first and last row whose elements are derived from (20).

The values of T_{ij} and S_{ij} can easily be calculated by splitting their integrals over all the elements as shown in (22)

$$\int_0^1 f\left(x\right)dx=\sum_{i=1}^{N}\int_{x_i}^{x_{i+1}} f\left(x\right)dx \qquad (22)$$

As the basis functions only have a maximum of two elements on which they are non-zero, the values of T_{ij} and S_{ij} can be calculated by just integrating over the few elements on which both basis functions are non-zero. This allows us to calculate the following coefficients

$$\begin{aligned} T_{i,i} &= \int_{x_{i-1}}^{x_i} \phi_i(x)^2 dx + \int_{x_i}^{x_{i+1}} \phi_i(x)^2 dx = \frac{2}{3}h, \\ T_{i,i-1} &= \int_{x_{i-1}}^{x_i} \phi_i(x)\phi_{i-1}(x) dx = \frac{1}{6}h, \\ S_{i,i} &= \int_{x_{i-1}}^{x_i} \left(\frac{\partial \phi_i}{\partial x}(x)\right)^2 dx + \int_{x_i}^{x_{i+1}} \left(\frac{\partial \phi_i}{\partial x}(x)\right)^2 dx = \frac{2}{h} \quad \text{and} \\ S_{i,i-1} &= \int_{x_{i-1}}^{x_i} \frac{\partial \phi_i}{\partial x}(x)\frac{\partial \phi_{i-1}}{\partial x}(x) dx = -\frac{1}{h}. \end{aligned} \tag{23}$$

For all other values, the basis functions do not overlap on non-zero elements and thus their product is zero. The result is a second-order linear differential equation with constant coefficients as given in(21). The solution to this equation can then be used in the finite element solution to get a numerical solution to the one-dimensional damped wave equation.

Time discretization

A first and second-order approximation is given by *central difference* [6] formula in (24). Substituting (24) in (21) gives (25)

$$\begin{aligned} \dot{U}(t) &\approx \frac{U(t+dt) - U(t-dt)}{2dt} \\ \ddot{U}(t) &\approx \frac{U(t+dt) - 2U(t) + U(t-dt)}{dt^2} \end{aligned} \tag{24}$$

$$T\left(\frac{U(t+dt) - 2U(t) + U(t-dt)}{dt^2}\right) + \xi\left(\frac{U(t+dt) - U(t-dt)}{2dt}\right) = c^2 S U(t) \tag{25}$$

For the special case where $\xi = 0$, after some algebraic manipulation the expression in (26) is obtained

$$U(t+dt) = -c^2 dt^2 T^{-1} S U(t) + 2U(t) - U(t-dt) \tag{26}$$

As the initial values are $U(0)$ and $\dot{U}(0)$ can use a first-order approximation to do the first step as given in (27) and use equation (26) for further iterations.

$$U(dt) = U(0) + dt\dot{U}(0) \tag{27}$$

Notice that (25) requires the inverse of the sparse (or more specific tridiagonal) matrix T and its inverse is generally not sparse.

The solution to (26) is solved for different element number and compared with the analytical solution (with both ends fixed) as given in (28). Also, time is varied and solutions are compared.

$$u(x,t) = \sum_{n=1}^{\infty}\left(A_n \cos\frac{an\pi t}{L} + B_n \sin\frac{an\pi t}{L}\right)\sin\frac{n\pi x}{L} \tag{28}$$

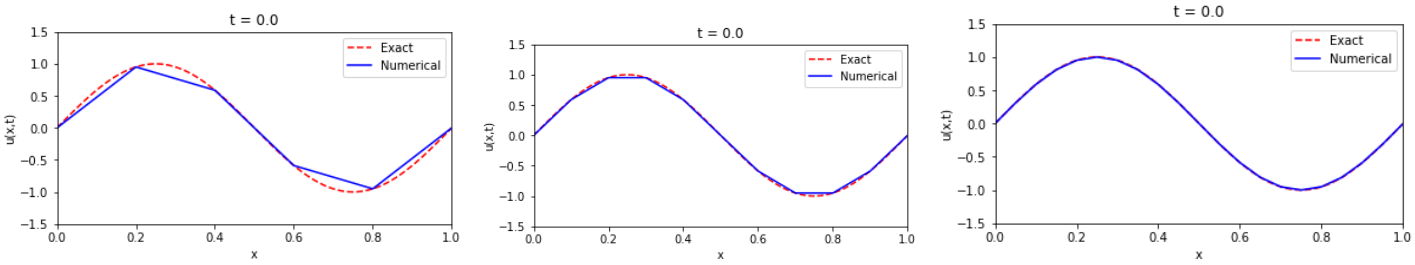
The particular solutions to (28) can be found with (29) and (30)

$$A_n = \frac{2}{L}\int_0^L f(x)\sin\frac{n\pi x}{L} dx \tag{29}$$

$$B_n = \frac{2}{an\pi}\int_0^L g(x)\sin\frac{n\pi x}{L} dx \tag{30}$$

Simulation Results

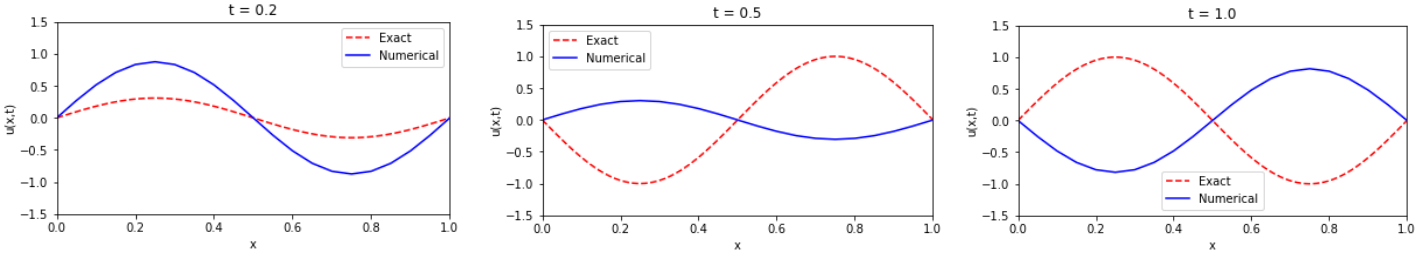
It is seen in Fig. 3a that with 5 elements the numerical solution differs significantly from the exact one at several points. When the number of elements was increased to 10 and the simulation repeated, we could see in Fig. 3b that both the discrepancy that was very pronounced in Fig. 3a was reduced significantly. By the time we increased the size of the element to 20, the exact and numerical solution was almost the same at all points. This is depicted in Fig. 3c. these results support the theory that a finer mesh will certainly give a numerical result that is very close to the analytical one.



(a) 5 (b) 10 (c) 20

Fig 3: Simulation with different number of elements

Since we now know that the optimal number of elements for the simulation is 20, we will proceed by retaining this number of elements and experiment with different time. It can be seen from Fig. 4 hat the smaller value of *t* gives a better approximation to the analytical solution.



(a) (b) (c)

Fig 4: Simulation with different time for a fixed number of elements

References

1. Hyperbolic partial differential equation (accessed, 28/12/2020) https://en.wikipedia.org/wiki/Hyperbolic_partial_differential_equation
2. Zachary S Tseng (2012). Second-Order Linear Partial Differential Equations Part IV https://fdocuments.in/document/second-order-linear-partial-differential-equations-part-iv-pt4pdfaa-second.html
3. Mathew P. Coleman (2013). Partial Differential Equations with MATLAB, Second Edition. Tailor & Francis Group. ISBN-13: 978-1-4398-9847-5
4. Louise Olsen-Kettle (2010). Numerical solution of partial differential equations. The University of Queensland School of Earth Sciences Centre for Geoscience Computing. ISBN: 978-1-74272-149-1
5. M. G. Larson and F. Bengzon (2010). The Finite Element Method: Theory, Implementation, and Applications. Springer.
6. William Bober (2014). Introduction to Numerical & Analytical Methods with MATLAB for Engineers and scientists. ISBN-13: 978-1-4665-7609-4

Python Code

```
# imports
import matplotlib.pyplot as plt;
import numpy as np;
import numpy.linalg as linalg;
import math;
from ipywidgets import *

# Amount of elements
N = 10;
# Amount of nodes
n = N + 1;
# Element size
h = 1.0/N;

# Constant
c = 0.4;
# Delta time
dt = 0.001;
# Amount of iterations
iterations = 20000;
# Stepsize
stepSize = 100;


# Matrix construction
# Time coefficient matrix
T = np.zeros((n, n));
T[0, 0] = 1; # Left boundary
T[N, N] = 1; # Right boundary
for i in range(1, N):
    for j in range(0, N+1):
        if(i==j):
            T[i, j] = (2.0/3.0)*h;
        if(abs(i-j) == 1):
            T[i, j] = (1.0/6.0)*h;
# Space coefficient matrix
S = np.zeros((n, n));
for i in range(1, N):
    for j in range(0, N+1):
        if(i==j):
            S[i, j] = (2.0/h);
        if(abs(i-j)== 1):
            S[i, j] = -(1.0/h);
# A single time step
def iteration(v, vDer):
    vNew = v + dt*vDer;
    q = -c*c*S@v;
    r = linalg.solve(T, q);
    vDerNew = vDer + dt*r;
    return (vNew, vDerNew);

# The real solution
def realU(x, t):
    return np.cos(2*np.pi*t)*np.sin(2*np.pi*x);
# The initial value of the finite element problem
u = np.zeros((n, 1));
uDer = np.zeros((n, 1));
for i in range(0, n):
    x = i*h;
    u[i] = realU(x, 0);

# Real solution
# The real solution
def realU(x, t):
    return np.cos(2*np.pi*t)*np.sin(2*np.pi*x);

# The initial value of the finite element problem
u = np.zeros((n, 1));
uDer = np.zeros((n, 1));
for i in range(0, n):
    x = i*h;
    u[i] = realU(x, 0);

# Interactive simulation
data = [];
data.append(u);
spacing = np.linspace(0.0, 1.0, n);
bigspacing = np.linspace(0.0, 1.0, 100);
for i in range(0, iterations):
    (uNew, uDerNew) = iteration(u, uDer);
    u = uNew;
    uDer = uDerNew;
    data.append(u);
def animation(t):
    plt.rcParams["figure.figsize"] = (6,3);
    axes = plt.gca();
    axes.set_ylim([-1.5,1.5]);
    axes.set_xlim([0,1]);
    plt.title("t = " + str(t*dt));
    plt.ylabel("u(x,t)");
    plt.xlabel("x");
    plt.plot(bigspacing, realU(bigspacing, t*dt), "r--", label='Exact');
    plt.plot(spacing, data[t], "b-", label='Numerical');
    plt.legend()
interact(animation, t = widgets.IntSlider(min = 0, max = iterations, step = stepSize, value = 0));

# Static plot

plt.rcParams["figure.figsize"] = (3,12)

timespace = [0, (int)(1.0/(3.0*dt)), (int)(2.0/(3.0*dt)), (int)(1.0/dt), (int)(3.25/dt)];

print(timespace);

plt.subplots_adjust(hspace=.0);

ax1 = plt.subplot("511");
ax1.set_ylim([-1.1,1.1]);
ax1.set_xlim([0,1]);
plt.plot(bigspacing, realU(bigspacing, timespace[0]*dt), "r--", label='actual');
plt.plot(spacing, data[timespace[0]], "bo");
plt.plot(spacing, data[timespace[0]], "b-", label="numerical");
plt.text(-0.55,0,"t="+str(dt*timespace[0]));
plt.text(-0.25,0.1,"u(x,t)", rotation=90);
plt.text(0.475, 1.4, "x");
plt.legend(bbox_to_anchor=(0.5, 1.2, 0., 0.1), loc=3,
      ncol=2, mode="expand", borderaxespad=0.)
ax1.xaxis.tick_top();

ax2 = plt.subplot("512", sharex=ax1);
ax2.set_ylim([-1.1,1.1]);
ax2.set_xlim([0,1]);
plt.plot(bigspacing, realU(bigspacing, timespace[1]*dt), "r--");
plt.plot(spacing, data[timespace[1]], "bo");
plt.plot(spacing, data[timespace[1]], "b-");
plt.text(-0.55,0,"t="+str(dt*timespace[1]));
plt.text(-0.25,0.1,"u(x,t)", rotation=90);
plt.setp(ax2.get_xticklabels(), visible=False);

ax3 = plt.subplot("513", sharex=ax1);
ax3.set_ylim([-1.1,1.1]);
ax3.set_xlim([0,1]);
plt.plot(bigspacing, realU(bigspacing, timespace[2]*dt), "r--");
plt.plot(spacing, data[timespace[2]], "bo");
plt.plot(spacing, data[timespace[2]], "b-");
plt.text(-0.55,0,"t="+str(dt*timespace[2]));
plt.text(-0.25,0.1,"u(x,t)", rotation=90);
plt.setp(ax3.get_xticklabels(), visible=False);

ax4 = plt.subplot("514", sharex=ax1);
ax4.set_ylim([-1.1,1.1]);
ax4.set_xlim([0,1]);plt.plot(bigspacing, realU(bigspacing, timespace[3]*dt), "r--");
plt.plot(spacing, data[timespace[3]], "bo");
plt.plot(spacing, data[timespace[3]], "b-");
plt.text(-0.55,0,"t="+str(dt*timespace[3]));
plt.text(-0.25,0.1,"u(x,t)", rotation=90);
plt.setp(ax2.get_xticklabels(), visible=False);
plt.setp(ax4.get_xticklabels(), visible=False);

ax5 = plt.subplot("515", sharex=ax1);
ax5.set_ylim([-1.1,1.1]);
ax5.set_xlim([0,1]);plt.plot(bigspacing, realU(bigspacing, timespace[4]*dt), "r--");
plt.plot(spacing, data[timespace[4]], "bo");
plt.plot(spacing, data[timespace[4]], "b-");
plt.text(-0.55,0,"t="+str(dt*timespace[4]));
plt.text(-0.25,0.1,"u(x,t)", rotation=90);
```